SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5117
NAME: G. BALASUBRAMANIAN
REG.NO:192521001
DEPARTMENT: B.TECH. IT
ASSIGNMENT: C03 – AT2
FACULTY NAME: DR. R. DHARANI

1) **Case Study: Security Weakness of MD5 Password Hashing and Migration to SHA-Based Hashing**

Title

Case Study on the Weaknesses of MD5 Password Hashing and the Adoption of Secure SHA-Based Hashing Techniques

Background

A software company stores user passwords in its database using the MD5 (Message Digest Algorithm 5) hashing algorithm. The company believes that storing hashed passwords instead of plain-text passwords provides sufficient security. However, after a cyberattack, hackers gain access to the password database and successfully recover many user passwords, leading to unauthorized account access and data theft.

Problem Statement

The attackers exploited the weaknesses of the MD5 hashing algorithm to crack user passwords quickly. Since many users reused common passwords, the attackers were able to reverse the hashes using modern password-cracking techniques.

Weaknesses of MD5

1. Fast Hashing Speed

MD5 is designed to generate hashes very quickly. Modern GPUs and specialized hardware can compute billions of MD5 hashes every second, making brute-force attacks highly effective.

2. Vulnerability to Rainbow Table Attacks

If passwords are stored without a unique salt, attackers can compare MD5 hashes against precomputed rainbow tables to recover passwords almost instantly.

3. Collision Vulnerability

MD5 is susceptible to collision attacks, where two different inputs produce the same hash value. Although collisions do not directly reveal passwords, they demonstrate that MD5 is no longer cryptographically secure.

4. Lack of Salting

The company stored passwords using plain MD5 hashing without adding unique salts. As a result, identical passwords produced identical hashes, allowing attackers to identify users sharing the same password.

5. Outdated Security Standard

MD5 has been considered insecure for many years and is no longer recommended for password storage by cybersecurity experts or security organizations.

Impact of the Security Breach

- **User passwords were successfully cracked.**
- **Unauthorized users accessed customer accounts.**
- **Sensitive customer information was exposed.**
- **The company's reputation suffered significant damage.**
- **Financial losses occurred due to recovery efforts and legal compliance.**
- **Customers lost trust in the company's security practices.**

Proposed Secure Hashing Approach Using SHA-Based Techniques

The company should replace MD5 with a more secure password storage mechanism based on the SHA-256 algorithm combined with additional security measures.

Recommended Approach

1. **Use SHA-256 instead of MD5.**
2. **Generate a unique random salt for every user.**
3. **Append or prepend the salt to the password before hashing.**
4. **Perform multiple hashing iterations (key stretching) to increase computational cost.**

5. **Store both the salt and the final hash securely in the database.**
6. **Rehash existing passwords when users log in after the migration.**

Example

MD5 Password Storage

Password: admin123

MD5 Hash:

0192023a7bbd73250516f069df18b500

Attackers can quickly recover this using rainbow tables.

SHA-256 with Salt

Password: admin123

Salt: X7@P9L!2

SHA-256(Salt + Password):

8d5a0d7e2d2b3d9d7d62d8d53c2e5b7cf8f3d2c6d8e2b6d9c7a1e5f8b4d2a9c3

Even if two users choose the same password, different salts generate different hashes, making attacks much more difficult.

Benefits of SHA-256 with Salting

- **Stronger cryptographic security than MD5.**
- **Protects against rainbow table attacks.**
- **Makes brute-force attacks significantly more time-consuming.**
- **Produces unique hashes for identical passwords because of random salts.**
- **Better aligns with modern security recommendations and industry practices.**

Comparison of MD5 and SHA-256

Feature	MD5	SHA-256
Hash Length	128 bits	256 bits

Feature	MD5	SHA-256
Security Level	Weak	Strong
Collision Resistance	Poor	Very High
Rainbow Table Protection	No (without salt)	Yes (with salt)
Recommended for Password Storage	No	Yes (with salting and iterations)
Current Industry Usage	Obsolete	Widely Used

Conclusion

The company's use of MD5 exposed user passwords to modern password-cracking techniques because of its fast hashing speed, collision vulnerabilities, and lack of salting. Migrating to SHA-256 with unique salts and multiple hashing iterations greatly improves password security by making brute-force and rainbow table attacks far more difficult. For even stronger protection, modern password hashing algorithms such as bcrypt, scrypt, or Argon2 are recommended, as they are specifically designed for secure password storage and provide better resistance against hardware-accelerated attacks.

Outcome: Replacing MD5 with SHA-256 (or preferably bcrypt/Argon2) significantly enhances the company's cybersecurity posture, protects user credentials, and reduces the risk of future password compromise.

2) **Case Study: Flaws in Digital Signature Implementation and Secure Signature Standards in Banking Applications**

Background

A banking application uses **digital signatures** to verify the authenticity and integrity of online transactions. Every transaction is digitally signed before being processed to ensure that it originates from the legitimate account holder. However, several customers report that unauthorized transactions have been approved even though they never authorized them. The bank launches a security investigation to identify weaknesses in the digital signature system.

Problem Statement

Despite using digital signatures, attackers were able to execute unauthorized financial transactions. This indicates flaws in the implementation, key management, or verification process rather than in the concept of digital signatures itself.

Possible Flaws in the Digital Signature Implementation

1. Weak Private Key Protection

Users' private keys may have been stored on insecure devices or transmitted over the network, allowing attackers to steal and misuse them.

2. Inadequate Authentication

The application may allow transactions to be signed without strong user authentication, making it easier for attackers to access user accounts.

3. Use of Weak Cryptographic Algorithms

The system may rely on outdated algorithms such as SHA-1 or weak RSA key sizes, making signatures vulnerable to cryptographic attacks.

4. Improper Signature Verification

The banking server may verify only part of the transaction data instead of the complete transaction, allowing attackers to modify transaction details without invalidating the signature.

5. Replay Attacks

If transaction IDs, timestamps, or nonces are not included in the signed data, attackers can capture valid signed transactions and replay them later.

6. Lack of Certificate Validation

Failure to verify digital certificates properly may allow attackers to use expired, forged, or revoked certificates.

7. Software Vulnerabilities

Programming errors, insecure APIs, or improper implementation of cryptographic libraries can weaken the effectiveness of digital signatures.

Impact of the Security Issue

- Unauthorized financial transactions were successfully processed.
- Customers experienced monetary losses.
- Customer confidence in the banking application decreased.
- The bank faced regulatory compliance issues.
- Financial and reputational damage occurred due to fraud investigations and compensation.

Recommended Improvements Based on Secure Signature Standards

1. Use Strong Digital Signature Algorithms

Implement secure and widely accepted algorithms such as:

- RSA-3072 or RSA-4096
- ECDSA (Elliptic Curve Digital Signature Algorithm)
- Ed25519 (where supported)

2. Use Secure Hash Functions

Replace weak hash functions with:

- SHA-256

- SHA-384
- SHA-512

These provide strong resistance against collision attacks.

3. Protect Private Keys

- Store private keys inside Hardware Security Modules (HSMs) or secure cryptographic tokens.
- Never transmit or store private keys in plain text.
- Encrypt keys using strong encryption standards.

4. Implement Multi-Factor Authentication (MFA)

Require additional authentication methods such as:

- One-Time Passwords (OTP)
- Biometric authentication
- Hardware security keys
- Mobile authenticator applications

5. Sign Complete Transaction Details

Include all critical information in the digital signature:

- Sender account
- Receiver account
- Transaction amount
- Date and time
- Transaction ID
- Currency
- Nonce or unique session identifier

Any modification should invalidate the signature.

6. Prevent Replay Attacks

Use:

- Unique transaction IDs
- Nonces

- Timestamps
- Session validation

These measures ensure each signed transaction can be processed only once.

7. Validate Digital Certificates

Regularly verify:

- Certificate validity period
- Certificate Authority (CA)
- Certificate Revocation List (CRL)
- Online Certificate Status Protocol (OCSP)

Reject expired or revoked certificates.

8. Perform Regular Security Audits

Conduct:

- Penetration testing
- Code reviews
- Cryptographic implementation audits
- Security monitoring and logging

Secure Transaction Verification Process

1. User authenticates using MFA.
2. Transaction details are generated.
3. SHA-256 computes the transaction hash.
4. User's private key signs the hash using RSA or ECDSA.
5. Bank verifies the signature using the user's public key.
6. Certificate validity is checked.
7. Transaction ID and timestamp are verified to prevent replay attacks.
8. If all checks succeed, the transaction is approved.

Comparison of Weak vs Secure Digital Signature Implementation

Feature	Weak Implementation	Secure Implementation
Signature Algorithm	SHA-1 / Weak RSA	RSA-3072, RSA-4096, ECDSA, Ed25519
Hash Function	MD5 / SHA-1	SHA-256 / SHA-384 / SHA-512
Private Key Storage	Plain software storage	HSM or Secure Key Store
User Authentication	Password only	Multi-Factor Authentication
Replay Protection	Not implemented	Nonces, Timestamps, Transaction IDs
Certificate Validation	Limited	Full CRL and OCSP Verification
Transaction Integrity	Partial verification	Complete transaction signed

---

Conclusion

The unauthorized banking transactions resulted from weaknesses in the implementation of the digital signature system rather than the digital signature concept itself. Poor private key protection, weak cryptographic algorithms, inadequate certificate validation, incomplete signature verification, and lack of replay protection allowed attackers to exploit the system. By adopting secure standards such as **RSA-3072/ECDSA with SHA-256 or stronger**, protecting private keys, implementing multi-factor authentication, validating certificates, and signing complete transaction details, the bank can significantly improve transaction security, prevent fraud, and restore customer trust.

3) **Case Study: Replay Attack on Kerberos Authentication and Secure Enhancements**

Background

A large enterprise uses the **Kerberos authentication protocol** to provide secure access to its internal network, applications, and file servers. Kerberos authenticates users through a trusted **Key Distribution Center (KDC)** without transmitting passwords over the network. Despite this, attackers successfully perform a **replay attack** and gain unauthorized access to enterprise resources.

Problem Statement

Attackers intercepted valid Kerberos authentication messages and replayed them to the server, allowing unauthorized access. Although Kerberos is designed to resist replay attacks, weaknesses in its implementation and configuration enabled the attack.

How the Replay Attack Could Occur

1. Interception of Authentication Tickets

An attacker captures a valid **Ticket Granting Ticket (TGT)** or **Service Ticket** while it is transmitted over the network.

2. Reuse of Captured Tickets

If the captured ticket is still valid and the server does not properly verify timestamps or authenticators, the attacker can resend (replay) the ticket to gain access.

3. Clock Synchronization Failure

Kerberos depends on synchronized system clocks. If there is excessive clock skew or poor time synchronization, replayed tickets may still be accepted.

4. Long Ticket Lifetime

If tickets remain valid for long periods, attackers have more time to reuse stolen credentials before they expire.

5. Weak Session Protection

If session keys or tickets are exposed through malware, insecure storage, or network compromise, attackers can impersonate legitimate users.

6. Improper Server Validation

The server may fail to verify the uniqueness of authenticators or reject duplicate authentication requests.

Impact of the Attack

- Unauthorized access to enterprise resources.
- Theft of confidential business information.
- Privilege escalation within the network.
- Financial losses and operational disruption.
- Violation of security policies and compliance requirements.
- Reduced trust in the organization's authentication system.

Recommended Enhancements to Prevent Replay Attacks

1. Enforce Strict Timestamp Validation

- Validate timestamps on every authentication request.
- Reject requests outside the allowed time window (typically ±5 minutes).

2. Maintain Accurate Clock Synchronization

- Synchronize all clients, servers, and the KDC using **Network Time Protocol (NTP)**.
- Monitor clock drift regularly.

3. Use Short Ticket Lifetimes

- Reduce the validity period of Ticket Granting Tickets (TGTs) and Service Tickets.
- Require periodic re-authentication for long user sessions.

4. Verify Authenticators

- Ensure every authentication request contains a fresh authenticator with a timestamp.
- Reject duplicate authenticators that have already been processed.

5. Protect Session Keys

- Encrypt session keys using strong cryptographic algorithms.
- Store keys securely in protected memory and avoid exposing them to unauthorized applications.

6. Enable Mutual Authentication

- Ensure both the client and the server authenticate each other.
- Prevent attackers from impersonating legitimate servers.

7. Monitor and Detect Suspicious Activity

- Log authentication events.
- Detect repeated authentication attempts using identical tickets.
- Generate alerts for unusual login patterns or multiple replay attempts.

8. Implement Multi-Factor Authentication (MFA)

Require an additional authentication factor such as:

- One-Time Password (OTP)
- Biometric verification
- Hardware security token
- Authenticator application

Even if a ticket is captured, attackers cannot complete authentication without the second factor.

Improved Kerberos Authentication Process

1. User logs in using secure credentials.
2. KDC issues a Ticket Granting Ticket (TGT).
3. Client requests a Service Ticket from the KDC.
4. The client sends the Service Ticket along with a **new authenticator containing a current timestamp**.
5. The server verifies:
    - Ticket validity

- Timestamp freshness

6. Mutual authentication is completed.
7. Access is granted only if all verification checks succeed.

Comparison of Existing and Improved Authentication

Feature	Existing System	Improved System
Ticket Lifetime	Long validity	Short validity
Timestamp Verification	Weak	Strict validation
Clock Synchronization	Inconsistent	Accurate NTP synchronization
Authenticator Validation	Limited	Duplicate detection enabled
Session Key Protection	Basic	Strong encryption and secure storage
Mutual Authentication	Optional	Mandatory
Multi-Factor Authentication	Not used	Enabled
Replay Attack Resistance	Low	High

Conclusion

The replay attack succeeded because attackers were able to reuse valid Kerberos authentication messages before they expired. Weak timestamp validation, long ticket lifetimes, poor clock synchronization, and inadequate authenticator verification increased the risk of unauthorized access. By enforcing **strict timestamp checks**, **short-lived tickets**, **accurate NTP synchronization**, **mutual authentication**, **secure session key management**, and **multi-factor authentication**, the enterprise can significantly strengthen its Kerberos authentication process and effectively protect against replay attacks.

Outcome: Implementing these improvements enhances the security of the Kerberos authentication system, minimizes replay attack risks, and ensures secure access to enterprise network resources.

4) **Case Study: Improving Message Authentication Using HMAC**

Title

Case Study on Message Integrity Failure and the Use of HMAC for Secure Message Authentication

Background

A secure communication system is designed to ensure that messages exchanged between a sender and a receiver remain unchanged during transmission. The system uses a **simple hash function (e.g., SHA-256)** to verify message integrity. The sender computes the hash of the message and sends both the message and its hash to the receiver. During a security audit, it is discovered that attackers are modifying messages and generating new hashes, allowing the altered messages to pass integrity verification.

Problem Statement

The communication system relies only on a hash function for message verification. Since hash functions are publicly known and require no secret key, attackers can modify a message, recompute its hash, and send both the modified message and the new hash to the receiver. As a result, the receiver cannot detect the tampering.

Why the Simple Hash Approach Fails

1. No Secret Key

A standard hash function does not use a secret key. Anyone can calculate the hash of a modified message.

2. Easy Hash Recalculation

After changing the message, an attacker simply computes a new hash and replaces the original one.

3. No Authentication

The receiver can verify only that the received hash matches the received message, not that the message came from the legitimate sender.

4. Vulnerability to Message Tampering

Since both the message and hash can be changed together, the integrity check becomes ineffective.

5. No Protection Against Active Attackers

Attackers intercepting communication can alter messages and generate valid hashes without knowing any secret information.

Impact of the Security Weakness

- Messages can be modified during transmission.
- Unauthorized commands may be accepted.
- Confidential business information may be altered.
- Financial transactions may be manipulated.
- Trust between communicating parties is compromised.
- Increased risk of fraud and data breaches.

Proposed Solution Using HMAC

To ensure both **message integrity** and **authentication**, the system should replace the simple hash with **HMAC (Hash-based Message Authentication Code)**.

What is HMAC?

HMAC combines:

- A cryptographic hash function (such as SHA-256)
- A shared secret key known only to the sender and receiver

The HMAC value can only be generated by someone possessing the secret key.

HMAC Authentication Process

1. The sender creates the message.
2. The sender combines the message with a secret key.
3. HMAC-SHA-256 computes the authentication code.
4. The sender transmits the message and HMAC.
5. The receiver uses the same secret key to recompute the HMAC.
6. If both HMAC values match, the message is accepted; otherwise, it is rejected.

Example

Weak Hash-Based Verification

Message:

Transfer ₹10,000 to Account A

SHA-256 Hash:

ABC123...

An attacker changes the message:

Transfer ₹1,00,000 to Account B

The attacker recalculates the SHA-256 hash and sends both the modified message and the new hash. The receiver accepts the altered message because the hash matches.

Secure HMAC Verification

Message:

Transfer ₹10,000 to Account A

Secret Key:

BankKey@2026

HMAC-SHA-256:

9A4F8D72E1C5...

If an attacker modifies the message, they **cannot generate the correct HMAC** without knowing the secret key. The receiver detects the mismatch and rejects the message.

Advantages of HMAC

- Ensures message integrity.
- Authenticates the sender using a shared secret key.
- Prevents message tampering.
- Resistant to replay and forgery when used with timestamps or nonces.
- Uses secure hash functions such as SHA-256 or SHA-512.

- Widely used in banking, APIs, VPNs, and secure communication protocols.

Comparison of Simple Hash and HMAC

Feature	Simple Hash	HMAC
Secret Key Required	No	Yes
Message Integrity	Yes	Yes
Sender Authentication	No	Yes
Prevents Message Tampering	No	Yes
Resistant to Forgery	No	Yes
Suitable for Secure Communication	No	Yes

Conclusion

The communication system failed because a simple hash function provides **integrity only**, not **authentication**. Since attackers can compute new hashes for modified messages, tampering goes undetected. Replacing the simple hash with **HMAC using SHA-256 (or SHA-512)** and a shared secret key ensures that only authorized parties can generate valid authentication codes. This protects against message modification, forgery, and unauthorized access, making the communication system significantly more secure.

Outcome: Implementing **HMAC-SHA-256** strengthens message authentication, guarantees data integrity, and prevents attackers from modifying messages without detection.

5) **Case Study: X.509 Certificate Trust Validation Errors and Secure Certificate Management**

Title

Case Study on X.509 Certificate Trust Validation Failures and Improvements for Secure Authentication

Background

A multinational organization uses **X.509 digital certificates** to secure communication between users, web servers, email systems, and internal applications through SSL/TLS. These certificates are issued by a trusted **Certificate Authority (CA)** and are intended to authenticate servers and encrypt communication. However, employees frequently receive **certificate trust validation errors**, preventing secure connections and disrupting business operations.

Problem Statement

Users encounter trust validation errors while accessing secure services. Investigation reveals weaknesses in certificate management, including expired certificates, incomplete certificate chains, and improper validation of Certificate Authority (CA) certificates.

Possible Issues in Certificate Management

1. Expired Certificates

The server may be using an expired X.509 certificate. Clients reject expired certificates because they are no longer considered valid.

2. Incomplete Certificate Chain

The server may not provide the required **intermediate CA certificates**, preventing clients from building a complete chain of trust to the trusted root CA.

3. Untrusted Certificate Authority

Certificates issued by an unknown or self-signed CA may not be trusted by client devices or browsers.

4. Incorrect Certificate Configuration

The certificate may contain an incorrect domain name or hostname that does not match the server being accessed, resulting in hostname validation failures.

5. Revoked Certificates

If a certificate has been revoked due to compromise or misuse, clients checking the **Certificate Revocation List (CRL)** or **Online Certificate Status Protocol (OCSP)** will reject it.

6. Weak Cryptographic Algorithms

Certificates using outdated algorithms such as **SHA-1** or weak RSA key sizes are considered insecure and may not be trusted by modern systems.

7. Improper Time Synchronization

If the client or server system clock is incorrect, certificates may appear as "not yet valid" or "expired," causing trust errors.

Impact of the Issue

- Users cannot establish secure connections.
- Business applications become inaccessible.
- Increased risk of man-in-the-middle (MITM) attacks.
- Loss of customer confidence in secure services.
- Operational delays and reduced productivity.
- Non-compliance with security standards and regulations.

Proposed Solutions for Proper Authentication and Trust Chain Validation

1. Use Certificates from Trusted Certificate Authorities

Obtain certificates from well-recognized and trusted Certificate Authorities (CAs) to ensure compatibility with operating systems and web browsers.

2. Install the Complete Certificate Chain

Configure servers to send:

- Server Certificate
- Intermediate CA Certificate(s)
- Root CA (where required)

This allows clients to verify the complete chain of trust.

3. Renew Certificates Before Expiration

- Monitor certificate expiry dates.
- Automate renewal processes.
- Replace certificates before they expire.

4. Enable Certificate Revocation Checking

Implement:

- Certificate Revocation List (CRL)
- Online Certificate Status Protocol (OCSP)

Reject revoked certificates immediately.

5. Use Strong Cryptographic Standards

Adopt secure cryptographic algorithms:

- SHA-256 or SHA-384
- RSA-3072 / RSA-4096
- Elliptic Curve Cryptography (ECC)

Avoid deprecated algorithms such as MD5 and SHA-1.

6. Verify Hostname Matching

Ensure the certificate's:

- Common Name (CN)
- Subject Alternative Name (SAN)

correctly matches the server's domain name.

7. Maintain Accurate System Time

Synchronize all systems using **Network Time Protocol (NTP)** so certificate validity periods are evaluated correctly.

8. Perform Regular Certificate Audits

Conduct periodic checks to identify:

- Expired certificates
- Misconfigured certificates

- Missing intermediate certificates
- Weak cryptographic algorithms
- Revoked certificates

Secure Certificate Validation Process

1. Client connects to the server.
2. Server sends its X.509 certificate along with intermediate CA certificates.
3. Client verifies:
    - Certificate validity period
    - Digital signature
    - Hostname (CN/SAN)
    - Certificate chain up to the trusted root CA
    - Revocation status using CRL or OCSP
4. If all validations succeed, a secure TLS session is established.
5. If any validation fails, the connection is rejected.

Comparison of Existing and Improved Certificate Management

Feature	Existing System	Improved System
Certificate Authority	Self-signed/Untrusted	Trusted Public CA
Certificate Chain	Incomplete	Complete Chain Installed
Certificate Renewal	Manual	Automated Renewal
Revocation Checking	Not Implemented	CRL and OCSP Enabled
Cryptographic Algorithm	SHA-1 / Weak RSA	SHA-256, RSA-3072/ECC
Hostname Verification	Inconsistent	Proper CN/SAN Matching

Feature	Existing System	Improved System
Time Synchronization	Poor	NTP Synchronization
Security Level	Low	High

Conclusion

The trust validation errors occurred due to poor certificate management practices, including expired certificates, incomplete trust chains, untrusted Certificate Authorities, hostname mismatches, weak cryptographic algorithms, and inadequate revocation checking. By implementing **trusted CA-issued certificates**, maintaining a **complete certificate chain**, enabling **CRL/OCSP validation**, using **strong cryptographic standards**, automating certificate renewal, and ensuring proper **time synchronization**, the organization can achieve reliable authentication, establish a valid chain of trust, and provide secure communication across its network.

Outcome: Proper X.509 certificate management ensures secure authentication, eliminates trust validation errors, protects against impersonation attacks, and strengthens the organization's overall cybersecurity posture.